

Projeto de Banco de Dados

Grupo: Akemi Almirante, Carlos Andrei Cícero, Lucas Schettini,
Luan Motta

Etapa 2: *triggers, views, stored procedures e ORM.*

1. Views:

Views são resultados virtuais de consultas em SQL, usadas para reduzir a repetição de consultas frequentes, armazenando na memória de sessão do SGBD. A requisição do projeto tinha a implementação das seguintes *views*:

- ***vw_pacientes_internados***: pacientes que estão atualmente internados (data_hora_saida IS NULL na internação mais recente).

Extremamente importante tanto no ambiente de trabalho quanto na administração do hospital em relação à disponibilidade. Foi implementada utilizando da consulta de procurar o paciente cuja data de saída ainda estava nula, como indicado.

- ***vw_residentes_sem_supervisor***: residentes que estão escalados em algum plantão, mas cujo preceptor não tem titulação de doutor (ou não possui supervisão ativa).

View importante para manter a ordem das escalas e evitar pontas soltas sobre os preceptores. Foi implementada usando de JOINS entre as relações de 'pessoa', 'preceptor', 'residente', 'plantao' e 'escala', procurando 'doutor' ou 'dr' usando ILIKE.

- ***vw_estatisticas_atendimentos_mensal***: agregação por mês e por unidade: total de atendimentos, média de duração, procedimentos mais comuns.

2. Triggers:

Os *Triggers* são importantes para a manutenção de regras de negócio, captura de erros e criação de logs. A requisição do projeto tinha a implementação dos seguintes *triggers*:

- **trg_check_sobreposicao_escala:** BEFORE INSERT/UPDATE na tabela ESCALA. Impede que um mesmo residente seja escalado no mesmo dia/turno em duas unidades diferentes.

Esse trigger é importante para evitar confusões na área de trabalho. Implementado utilizando uma consulta para filtrar as novas inserções em plantão e outra consulta para checar as condicionais de 'dia_semana', e 'id_unidade' em relação ao novo residente.

- **trg_audita_atendimento:** AFTER INSERT/UPDATE/DELETE em ATENDIMENTO. Registra em uma tabela AUDITORIA_ATENDIMENTO (id_auditoria, id_atendimento, operação, usuario, data_hora, dados_antigos (JSON), dados_novos (JSON)).

Esse trigger é um ponto de log muito importante. Implementado usando a condicional 'TG_OP' = 'x', em que achamos na documentação que TG_OP armazena a operação que está sendo realizada no pipeline da função atual. Os dados que deveriam ser salvos em JSON o foram utilizando a função 'row_to_json()'.

- **trg_atualiza_media_procedimentos:** AFTER INSERT em PROCEDIMENTO_REALIZADO. Atualiza uma coluna media_tempo_procedimento na tabela PROCEDIMENTO (média do tempo_real_minutos daquele procedimento em todos os atendimentos).

Uso de trigger para automatizar a captura de valores estatísticos. Foi implementado com uma consulta para armazenar a média antiga e um update para atualizar o valor novo. Também há um NOTICE como log da variação de média.

3. Stored Procedures:

Procedures são subprogramas armazenados no banco de dados que executam um conjunto de comandos pré-definidos de forma reutilizável. São utilizadas principalmente para garantir que operações relacionadas sejam executadas de forma consistente, além de facilitar o trabalho de manutenção e atualização da aplicação.

Nessa etapa, foram solicitadas as seguintes procedures:

- **sp_registrar_atendimento_completo:** Essa procedure recebe os dados de um atendimento juntamente com uma lista de procedimentos realizados, enviada no formato JSON. Ela cadastra o atendimento e todos os seus procedimentos em uma

só operação. Sua importância se dá pelo fato de que evita que um atendimento seja registrado de forma incompleta, e além disso, são feitas verificações para confirmar a existência do paciente, residente, preceptor e unidade. Como toda a execução ocorre dentro de uma transação, caso algum procedimento seja inválido ou alguma inserção venha a falhar, a operação inteira é desfeita.

- **sp_calcular_tempo_medio_espera**: Essa procedure calcula para cada unidade, o tempo entre a chegada e a saída do paciente, representada pela data e hora do atendimento e o início do primeiro procedimento realizado. Sua importância deve-se justamente ao fato que ela identifica o tempo médio de espera, informação necessária para a análise de eficiência de cada unidade e atrasos no atendimento. O resultado é armazenado em uma tabela temporária "resultado_tempo_medio_espera". Essa tabela existe apenas durante a sessão atual do banco.
- **sp_reajustar_escala**: Essa última procedure recebe o identificador de um residente, o dia e o turno atual de sua escala, e o novo dia e turno para os quais ele será transferido. A procedure verifica se o residente existe, se os turnos informados são válidos, se o residente possui escala no dia e turno antigos e se ele já está escalado no novo dia e turno. Caso alguma condição não seja atendida, a operação é cancelada.

ORM:

A ORM escolhida foi a recomendada pela especificação, o SQLAlchemy. Ele possui o suporte a PostgreSQL, que era a SGBD que estamos utilizando, e sua implementação via objetos era benéfica para relações de herança e para o aprendizado. Os objetivos base eram:

- Mapeamento objeto-relacional (classes/entidades)

Feito no arquivo orm/[models.py](#), em que há a "tradução" de relações SQL puro na sintaxe da SQLAlchemy. Com a estrutura de classes, foi usada herança quando necessário.

- Implementação das operações da Etapa 1

- Feito no arquivo orm/[CRUD.py](#).
- Uso de sessões/transações via ORM
 - Implementado em orm/[main.py](#), acessado quando se acessa a interface web do projeto.
- Consultas usando a DSL/filter da ORM (não SQL cru)
 - Feitas em orm/[CRUD.py](#) & orm/adv_queries.py. Tentamos fazer as consultas com o maior rigor possível.
- Relacionamentos (lazy loading vs eager loading)

Por padrão, o SQLAlchemy usa lazy loading. Para demonstrarmos isso, usamos a consulta 'listar_ultimo_atendimento' em orm/adv_queries.py com eager loading, foi um bom exemplo de sua eficácia:

- Lazy Loading: para cada atendimento retornado, a ORM dispara consultas separadas quando você acessa 'residente', 'preceptor', 'procedimentos_realizados'. Ele busca um-por-um nos vínculos, dada uma consulta. Para n consultas, temos $3*n + 1$ consultas "excessivas".
- Eager Loading: a ORM faz consultas adicionais logo depois da query principal, carregando todos os relacionamentos de uma vez. Logo, para n consultas, temos (numero_de_vínculos) consultas.

4.1: Consultas Analíticas:

As consultas analíticas foram construídas utilizando a DSL (Domain Specific Language) do SQLAlchemy para abstrair lógicas complexas de SQL diretamente no código Python. Foram implementadas três consultas principais focadas na extração de relatórios e métricas do sistema:

- **Preceptores que atenderam pacientes flamenguistas:** Esta consulta utiliza múltiplos *JOINS* sequenciais para conectar as tabelas Preceptor, Atendimento e Paciente. A filtragem ocorre via 'where(paciente.is_flamengo == True)' e utiliza o

método 'distinct()' para evitar que o mesmo preceptor apareça repetido caso tenha realizado vários atendimentos para pacientes com essa característica.

- **Último atendimento de cada paciente:** Para extrair apenas o registro mais recente, foi criada uma subconsulta utilizando a função de agregação 'func.max(atendimento.data_hora)' agrupada pelo id_paciente. Esta consulta também atua como a prova de conceito do *Eager Loading* mencionada no tópico de relacionamentos, empregando opções de 'selectinload' para pré-carregar de forma eficiente todas as dependências do atendimento (residentes, preceptores e procedimentos realizados);
- **Percentual de procedimentos de alto risco por residente:** Trata-se de uma análise estatística e de ranking que cruza cinco entidades relacionais (Residente, Atendimento, Procedimento_realizado, Procedimento e NivelRisco). A lógica de cálculo utiliza o operador lógico case nativo da ORM para isolar o nível 'ALTO'. Em seguida, operações matemáticas diretas no banco com 'func.sum()' e 'func.count()' estabelecem a proporção percentual de risco assumida por cada residente durante a execução de suas atividades.

4.2: Tratamento de Concorrência:

- O tratamento de concorrência evita inconsistências quando vários usuários acessam ou modificam os mesmos dados ao mesmo tempo. No sistema, são utilizadas transações com **COMMIT** e **ROLLBACK**, garantindo que uma operação seja concluída por inteiro ou totalmente desfeita em caso de erro.
- Também podem ser aplicados bloqueios de registros com **FOR UPDATE**, impedindo que duas transações alterem simultaneamente o mesmo atendimento ou a mesma escala. Além disso, restrições como **PRIMARY KEY**, **FOREIGN KEY** e **UNIQUE**, juntamente com as triggers, ajudam a evitar duplicações, conflitos de escala e relacionamentos inválidos.

